Mietverwaltung mit MongoDB und Redis

Projektdokumentation · Modul NoSQL-Datenbanken · Oct 1, 2026 · @Jiyan Barut

1 Problemstellung

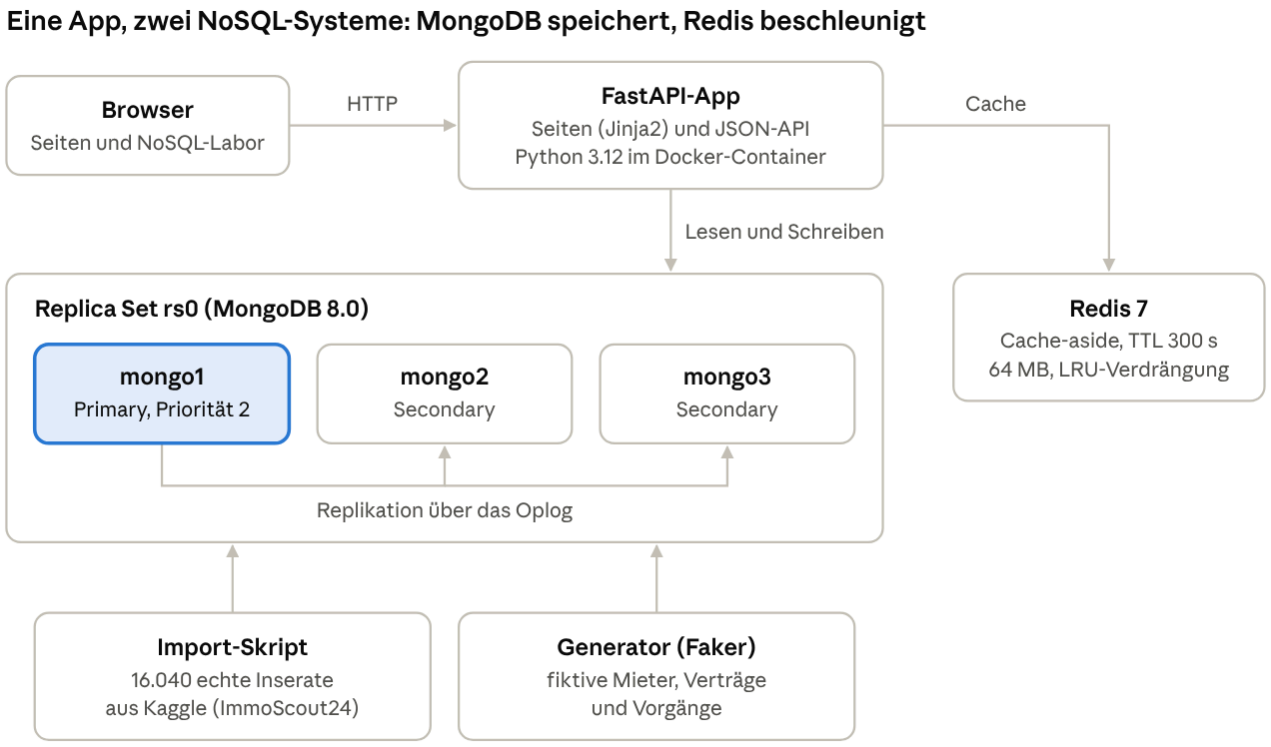
Ein Document Store passt zur Mietverwaltung überall dort, wo Daten als zusammenhängende, unterschiedlich aufgebaute und historisch wachsende Einheiten anfallen. Bei Beziehungen über viele Datensätze und bei Konsistenz zahlt man dafür einen Preis. Das Projekt prüft diese These an einem lauffähigen System.

Kleine Immobilienverwalter betreuen Objekte sehr unterschiedlicher Art: Eigentumswohnungen, Mehrfamilienhäuser mit Garagen und Stellplätzen, Wohn- und Geschäftshäuser mit Gewerbeeinheiten. Zu jedem Mietverhältnis gehören tief strukturierte Unterlagen wie Übergabeprotokolle mit Räumen, Mängeln und Zählerständen sowie eine Historie der Mietanpassungen. Dazu kommen Vorgänge wie Schäden, Mietminderungen, Kündigungen und Korrespondenz, die je nach Art andere Angaben brauchen.

Leitfrage: Warum ist ein Document Store, hier MongoDB, für diese Anwendung geeignet, und welche Vor- und Nachteile entstehen gegenüber einem relationalen Datenbanksystem? Ergänzend setzt das System Redis als Key-Value-Store ein. Alle Messwerte in Abschnitt 4 stammen aus dem laufenden System.

2 Architektur und Daten

Das System besteht aus einer FastAPI-Anwendung, einem MongoDB-Replica-Set mit drei Knoten und Redis; ein einziger Befehl (`docker compose up`) startet alles.



Architektur · Docker Compose, ein Befehl startet alle fünf Container

Die App spricht MongoDB über eine Verbindung mit allen drei Knoten an. Fällt der Primary aus, findet der Treiber den neuen selbst.

Collection	Herkunft	Dokumente
`angebote`	Echt: ImmoScout24-Inserate aus Baden-Württemberg (2018–2020), über Kaggle; 51 offensichtliche Erfassungsfehler aussortiert	16.040
`vorgaenge`	Fiktiv, aus praxisnahen Vorlagen (Wasserschaden, Mietminderung, Kündigung)	11.030
`mieter`	Fiktiv (Faker), Personen und Firmen	2.409
`vertraege`	Fiktiv, aufgebaut auf den echten Mieten und Flächen	2.063
`objekte`	Adresse, Baujahr und erste Wohnung echt, weitere Einheiten abgeleitet	250 mit 990 Einheiten

Insgesamt sind es rund 31.800 Dokumente. Personendaten sind bewusst fiktiv: Echte Mieterdaten wären auch mit gekürzten Namen noch personenbezogen im Sinne der DSGVO. Der Kaggle-Datensatz ist nur für Forschungszwecke freigegeben und liegt deshalb nicht im Repository. Ein fester Zufallswert sorgt dafür, dass jeder Import denselben Datenstand erzeugt.

3 Datenmodell

Das Modell bettet ein, was zusammen gelesen und geändert wird, und referenziert, was eigenständig existiert oder in einer n:m-Beziehung steht.

Collection	Inhalt	Entscheidung	Begründung
`objekte`	Adresse, Eigentümer, `einheiten[]`	Einbetten	Einheiten existieren nie ohne ihr Objekt und werden mit ihm gelesen. Wohnung, Gewerbe, Garage und Stellplatz haben jeweils eigene Felder.
`vertraege`	Konditionen, `mietanpassungen[]`, `uebergaben[]`	Einbetten und Referenz	Historie und Übergabeprotokolle gehören zum Vertrag; ein Lesezugriff liefert alles (größter Vertrag ca. 4 KB). Einheit und Mieter werden referenziert.
`mieter`	Personen und Firmen	Referenz über `mieter_ids[]`	Ein Mieter hat über die Zeit mehrere Verträge, ein Vertrag mehrere Mieter (n:m).
`vorgaenge`	sechs Typen, z. B. Schaden, Kündigung, Frist	Eigene Collection, polymorph	Vorgänge wachsen unbegrenzt. Eingebettet würde das Objekt-Dokument stetig größer (Grenze 16 MB je Dokument).
`angebote`	echte Inserate	Unverändert übernommen	Jedes Inserat speichert nur die Angaben, die der Anbieter gemacht hat.

Gezielte Denormalisierung. Der Vertrag speichert die Mieternamen und die Objektbezeichnung als Kopie, damit Listen ohne Lookup auskommen. Auch der Status „vermietet/leer“ steht doppelt, am Vertrag und an der eingebetteten Einheit. Der Preis dafür ist, dass jede Änderung alle Kopien finden muss (Abschnitt 4).

Polymorphie. Mieter (Person oder Firma), Einheiten (vier Typen) und Vorgänge (sechs Typen) teilen sich je eine Collection, jeder Typ mit eigenen Feldern. Relational wären dafür Vererbungstabellen oder viele leere Spalten nötig.

Regeln trotz Flexibilität. Jede Collection hat einen `$jsonSchema`-Validator mit Pflichtfeldern und erlaubten Werten. Für Mieter verlangt eine `oneOf`-Regel bei Personen Vor- und Nachname, bei Firmen den Firmennamen. MongoDB lehnt Verstöße serverseitig ab (Fehlercode 121).

4 NoSQL-Konzepte im System

Jedes Konzept ist im „NoSQL-Labor“ der Anwendung per Knopfdruck vorführbar; die Werte unten sind dort gemessen (lokal, Docker, 4 GB Arbeitsspeicher).

Konzept	Umsetzung	Messung
Replikation und Failover	Replica Set mit drei Knoten, `mongo1` mit Priorität 2	Primary hart beendet: neuer Primary nach 10,3 s, nächster Schreibzugriff in 10,6 ms ohne Neustart der App
Einstellbare Konsistenz	Write Concern und Read Preference je Abfrage	`w: 1` und Lesen vom Secondary: 48 von 50 Lesezugriffen sahen den alten Wert; `w: "majority"` und Lesen vom Primary: 0 von 50
Transaktionen	Mieterwechsel: fünf Schreibvorgänge in drei Collections	Abbruch nach Schritt 3: weder neuer Mieter noch neuer Vertrag existieren, alter Vertrag unverändert
Indizes	Compound-Index auf PLZ und Wohnfläche, Multikey-Indizes auf Arrays	Marktvergleich: 13 statt 16.040 gelesene Dokumente, 2 ms statt 49 ms
Caching mit Redis	Cache-aside mit 300 s TTL, Invalidierung bei jedem Schreibzugriff	Dashboard: 186,6 ms aus MongoDB, 0,3 ms aus Redis; ohne Invalidierung 89 statt 90 offene Schäden angezeigt

CAP-Einordnung. Mit `majority` verhält sich MongoDB wie ein CP-System: Während der Wahl nimmt kein Knoten Schreibzugriffe an, dafür gehen keine bestätigten Daten verloren. Mit `w: 1` und Lesen vom Secondary verschiebt sich die Wahl Richtung

Verfügbarkeit und Tempo, und Leser sehen veraltete Werte. Redis als Cache arbeitet bewusst mit verzögerter Konsistenz, begrenzt durch die TTL.

Aggregation. Das Dashboard verbindet vier Collections in fünf Pipelines, unter anderem mit `$facet` (vier Auswertungen in einem Durchlauf), `$lookup` und `$median`. Diese Abfragen sind der Grund für den Cache: sie sind die teuersten im System.

Sharding ist nicht umgesetzt. Rund 31.800 Dokumente passen problemlos in ein Replica Set, Sharding brächte nur Betriebsaufwand. Bei einer Plattform für viele Verwalter wäre eine Verwalter-ID der Shard Key, damit die Abfragen eines Verwalters auf einem Shard bleiben. Ein steigender Schlüssel wie das Datum wäre ungeeignet, weil alle neuen Vorgänge auf demselben Shard landen würden.

5 Vergleich mit einem relationalen System

Fünf von neun Kriterien sprechen für MongoDB, vier für ein relationales System wie PostgreSQL. Die Vorteile liegen bei Struktur und Betrieb, die Nachteile bei Integrität und Auswertung.

Kriterium	MongoDB	Relational	Im Vorteil
Uneinheitliche Daten	Fehlende Angaben entfallen; Inserate haben 7 bis 20 Felder	49 Spalten mit vielen NULL-Werten oder Vererbungstabellen	MongoDB
Vertrag mit Historie lesen	Ein Dokument, ein Zugriff	Fünf Tabellen (Vertrag, Mietanpassung, Übergabe, Zählerstand, Raum), vier Joins	MongoDB
Neue Anforderung, z. B. Ladestation an Garagen	Neues Feld sofort nutzbar, Validator anpassen	`ALTER TABLE` und Migration	MongoDB
Ausfallsicherheit	Replica Set mit automatischem Failover eingebaut	Meist über Zusatzwerkzeuge	MongoDB
Horizontale Skalierung	Sharding eingebaut (hier nicht nötig)	Aufwendig	MongoDB
Referenzielle Integrität	Sichert die Anwendung, z. B. kein Vertrag ohne Mieter	Fremdschlüssel erzwingen sie	Relational

Kriterium	MongoDB	Relational	Im Vorteil
Redundanz	Kopien müssen mitgepflegt werden (Namensänderung)	Normalisiert, jede Angabe an einer Stelle	Relational
Transaktionen	Möglich, aber nur mit Replica Set und teurer	Standard für jede Änderung	Relational
Übergreifende Auswertungen	Aggregation Pipeline, `$lookup` umständlich	SQL mit Joins und `GROUP BY`, ausgereift	Relational

In der Mietverwaltung überwiegen Lesezugriffe auf ganze Einheiten: ein Objekt mit seinen Einheiten, ein Vertrag mit seiner Historie. Genau diese Zugriffe bedient das Dokumentmodell mit einem Zugriff. Die relationalen Stärken zeigen sich im Dashboard, dessen Auswertungen über vier Collections die teuersten Abfragen sind und deshalb im Cache liegen.

6 Fazit

Ein Document Store eignet sich für die Mietverwaltung, weil ihre Kerndaten Dokumente sind: Objekte mit Einheiten, Verträge mit Historie und Protokollen, Vorgänge unterschiedlicher Art. Die echten Daten bestätigen das: 16.040 Inserate mit 7 bis 20 Feldern hätten relational vor allem leere Spalten erzeugt.

Der Preis ist Verantwortung in der Anwendung. Referenzielle Integrität und jede bewusst angelegte Kopie müssen per Code, Validator oder Transaktion abgesichert werden; das Experiment mit der Namensänderung zeigt, wie schnell eine vergessene Kopie veraltet. Übergreifende Auswertungen sind mit SQL einfacher.

Grenzen der Arbeit. Alle Messungen liefen lokal auf einem Rechner und lassen sich nicht auf echte Netzwerklatenz oder deutlich größere Datenmengen übertragen. Mieter, Verträge und Vorgänge sind aus Datenschutzgründen fiktiv, aufgebaut auf den echten Objektdaten.

Empfehlung. MongoDB für den operativen Kern, Redis für teure Auswertungen. Wächst der Bedarf an Berichten, gehört ein relationales oder analytisches System daneben, statt das Dokumentmodell dafür zu verbiegen (Polyglot Persistence).